Integrating Retrieval-Augmented Generation (RAG) and Knowledge Augmented Generation (KAG) Frameworks to Build Accurate Enterprise Question Answering Systems

Yonghe Guo[1], Longchuan Yan[1], Jianing Niu[1], Dequan Gao[1], Xiaoyu Yuan[2*]
1. State Grid Information&Telecommunication Branch, Beijing, China
2. Beijing Kedian Tuoyu Intelligent Technology Co., Ltd, Beijing, China
372687651@qq.com, lchyan@sgcc.com.cn, jianing-niu@sgcc.com.cn, gao_dequan@sgcc.com.cn, michaelyuanxiaoyu@qq.com
Corresponding Author: Xiaoyu Yuan Email: michaelyuanxiaoyu@qq.com

2025 IEEE 8th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC) | 979-8-3315-0951-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/IAEAC65194.2025.11166604

***Abstract*—With the rapid development of Large Language Models (LLMs), more and more requirements from enterprises for accurate question answering (QA) systems are emerging. LLMs have shown great potential in natural language processing (NLP) but face limitations such as hallucinations and lack of domain-specific knowledge. Retrieval-Augmented Generation (RAG) addresses these issues by retrieving external information to enhance LLMs' outputs, while Knowledge Augmented Generation (KAG) further improves reasoning capabilities through knowledge graphs. We propose five strategies for combining RAG and KAG, each balancing storage, retrieval efficiency, and response accuracy. Experiments using a dataset from a power industry company show that integrating RAG and KAG improves QA system performance, with Strategy 5 achieving the highest accuracy rate of 78%. Our findings highlight the potential of combining retrieval and knowledge augmentation to enhance enterprise QA systems.**

Keywords—Large Language Models; Question Answering; RAG; KAG

I. INTRODUCTION

In recent years, the rapid development of artificial intelligence technology has been remarkable, especially the great potential of large language models (LLMs) in natural language processing (NLP). As a type of artificial intelligence model based on deep learning, LLMs focus on the understanding, generation, and reasoning of natural language. With their massive scale of parameters, LLMs are trained on vast amounts of text data, learning the statistical patterns, semantic relationships, and knowledge of language, thereby acquiring powerful capabilities for text understanding and content generation [1]. LLMs are capable of performing a variety of complex tasks and have demonstrated excellence in language understanding and generation, as well as contextual learning. Researches show that LLMs have been widely applied in multiple industries, including healthcare, automotive, finance, education, and e-commerce [2].

Despite their excellent performance in numerous NLP tasks and significant achievements in various industry applications, LLMs still have some inherent limitations, such as hallucinations, incorrect information, lack of verifiability, lack of domain knowledge [1, 2, 20]. The problem of LLMs generating incorrect information or hallucinations is particularly prominent, especially when dealing with specialized knowledge or tasks that require high-precision information, such as medical diagnosis, legal consultation, and financial decision-making. This can lead to serious negative consequences. For example, research shows that when answering specific legal queries, the rate of fabricated content by LLMs ranges from 69% to 88% [21]. This issue greatly restricts the widespread application of LLMs in various industries.

The main reason for LLMs generating incorrect information or hallucinations is their dependence on training datasets, which prevents them from accessing the latest information. To address this issue, Retrieval-Augmented Generation (RAG) [10] has been proposed as an innovative solution. By using a retrieval module to supplement the training data of LLMs with the latest information from external sources, RAG can generate accurate answers. Recent studies have proven that RAG has shown great potential in knowledge-intensive tasks and has also performed excellently in general language tasks and various downstream applications [11, 12, 13, 14].

However, other studies [11, 13, 14, 20] have shown that RAG may still experience hallucinations when dealing with complex queries. To overcome the limitations of RAG and enhance its performance, knowledge graphs (KGs) based RAG has emerged. GraphRAG [22], based on the traditional RAG framework, strengthens the intrinsic connections between entities, communities, and text chunking, and

skillfully integrates the knowledge from existing KGs. Studies [22, 23] have shown that GraphRAG has improved the performance of RAG systems in multi-hop and cross-paragraph tasks.

Going a step further, to enhance the performance of RAG in specialized fields, Ant Group and other institutions have proposed an open-source framework for domain-specific knowledge enhancement service, KAG (Knowledge Augmented Generation) [27]. KAG enhances the question answering (QA) performance of LLMs in specific domains through knowledge augmentation, supports logical reasoning and multi-hop fact-based question answering, and significantly improves the accuracy and efficiency of reasoning and question answering. Studies [11, 13, 28] have shown that while RAG excels at enhancing the model's generation capabilities by retrieving external knowledge bases, KAG has unique advantages in knowledge representation, logical reasoning, and multi-hop question answering [27].

RAG and KAG are highly complementary, and integrating the two can leverage their strengths and compensate for their weaknesses, thereby improving the overall performance of enterprise QA systems. Modern enterprise-level QA systems need to have both strong retrieval capabilities and logical reasoning abilities to handle complex domain-specific problems. The integration of RAG and KAG can better meet these needs.

In this paper, we propose five strategies for integrating the RAG and KAG frameworks, aiming to fully utilize the strengths of RAG and KAG to improve the accuracy of enterprise QA systems in answering query questions.

II. Background and Related Works

A. Large Language Models (LLMs)

Large Language Models (LLMs) are neural network models with an extremely large number of parameters, capable of efficiently performing a variety of NLP tasks, including content generation, text classification, question answering, and language translation, among others [1, 2]. As the scale of model parameters continues to expand, LLMs have demonstrated remarkable improvements in language understanding and generation capabilities. Since the release of GPT-3 [4] in 2020, LLMs have quickly become a research hotspot in the field of artificial intelligence, giving rise to a series of advanced LLMs, such as GPT [4, 5], LLaMA [6, 7], Qwen [8] and DeepSeek [9].

LLMs are usually based on neural network architectures, particularly the Transformer architecture [3]. The core of the Transformer architecture is the Self-Attention Mechanism, which enables the model to dynamically focus on the relationships between different parts of the text during processing. The training of LLMs generally employs unsupervised learning, using large volumes of text data (such as books, web content, news articles, etc.) as training corpora. The training process begins with the collection of training corpora, followed by the use of word embedding tables to map words into specific vector spaces, allowing computers to process language in a numerical manner. With the aid of powerful computing resources, the models train their extensive parameters, continuously adjusting them during the training process to learn linguistic patterns and semantic information from the data, thereby achieving optimal performance in NLP tasks. Additionally, pre-trained LLMs typically require fine-tuning to better address downstream tasks such as text classification, question answering, document summarization, and text generation [1, 2].

However, LLMs also face numerous challenges and limitations [1]. First, the training of LLMs requires vast amounts of data, and the training data may be biased, leading to biased content generated by the models. For example, if the training data contains one-sided descriptions of certain groups, the content generated by the model may also exhibit similar biases. Second, although LLMs excel in language processing, they still cannot fully understand the complex emotions and implied meanings behind language as humans do. For example, when processing texts with sarcasm or puns, the models may misinterpret them. Moreover, training and running LLMs require substantial computing resources and storage space, making it difficult for some small businesses and individuals to access the most advanced LLMs. Despite the wide application of LLMs in many industries, further expansion and in-depth application require the resolution of the aforementioned issues.

B. Retrieval-Augmented Generation (RAG)

RAG [10] is an innovative technology that integrates information retrieval with LLMs, aiming to significantly enhance the quality and accuracy of the output from generative models.

The workflow of RAG is primarily divided into two stages: retrieval and generation [11, 12]. In the retrieval stage, the process begins with an input query. The retrieval module, based on the user's question or the text to be replied to, extracts relevant information from knowledge databases or external sources. Then these relevant information are ranked by rerank model based on their relevance to the input query. At last, the top k information are selected as the retrieval results. In generation stage, the generative model uses the input query and the retrieved information to generate a text response. The generated text may undergo additional post-processing steps to ensure its grammatical correctness and semantic coherence.

By leveraging its retrieval capabilities, RAG can provide the generative model with the latest facts and evidence, enabling it to quickly adapt to changes in new data and events. This effectively improves the accuracy

of the generated text and significantly reduces the issue of hallucinations. RAG provides factual basis for the retrieved results, and the generated answers can be annotated with source documents, which not only enhances the credibility of the responses but also improves their interpretability. Moreover, by connecting to text datasets specific to particular industries or fields, RAG can offer targeted professional knowledge support, precisely meeting the specialized needs of different tasks [13, 14] and has been widely applied in clinical predictions [15], legal QA [16], education [17, 18], finance [19] and other industries.

C. Graph Retrieval-Augmented Generation (GraphRAG)

GraphRAG [22] is an innovative knowledge retrieval enhancement technology that skillfully integrates the extensive knowledge representation capabilities of knowledge graphs with the powerful generative capabilities of LLMs. Compared to traditional RAG methods, GraphRAG leverages the strong retrieval capabilities of knowledge graphs and community hierarchical structures to achieve efficient and reliable retrieval of complex information, significantly enhancing the ability to process complex information and improving the accuracy of complex question answering [22, 23].

GraphRAG consists of two main processing stages: the indexing stage and the querying stage. During the indexing stage, the LLM is used to automatically construct the knowledge graph, extracting corresponding nodes, edges, and covariates. Community detection techniques are then applied to partition the entire knowledge graph into subgraphs, followed by bottom-up summarization and abstraction of these subgraphs using the LLM. In the querying stage, for a specific query, global search aggregates all relevant community summaries into a prompt, which is then used by the LLM to generate final answers.

As a new generation of RAG technology, GraphRAG significantly enhances the quality of generation and data processing efficiency by introducing knowledge graphs. With the continuous emergence of research findings such as GRAG [24], GNN-RAG [25], and HippoRAG [26], GraphRAG is demonstrating broad application potential in an increasing number of fields, including question answering systems, information extraction, biomedicine, academic research, and legal analysis.

D. Knowledge Augmented Generation (KAG)

KAG [27] is an advanced reasoning and question-answering framework jointly launched by Ant Group in collaboration with Zhejiang University and other institutions. Built upon the OpenSPG engine and LLMs, it aims to integrate the strengths of knowledge graphs and vector retrieval to provide more rigorous reasoning and question-answering capabilities, and significantly enhance the reasoning and generation capabilities of models in domain-specific question answering.

Compared to GraphRAG, KAG has further optimized and innovated on the basis of enhancing retrieval and reasoning with knowledge graphs. It proposes the LLM friendly knowledge representation framework LLMFriSPG, which enables mutual indexing between knowledge graphs and original text fragments, better integrating the advantages of graph structures with the richness of textual information. KAG also draws on the strengths of GraphRAG in graph reasoning and adds a hybrid reasoning engine guided by logical forms, thereby achieving more powerful reasoning capabilities and the ability to handle more complex multi-hop queries.

Research [11, 28, 29] indicates that RAG excels in open-domain tasks, being adept at handling dynamic unstructured data. In contrast, KAG has a clear advantage in scenarios that require obtaining factual and structured information from knowledge graphs. KAG is particularly suitable for professional fields that demand complex reasoning and multi-hop fact-based question answering, such as finance, healthcare, law, and government affairs. Research [27] shows that KAG achieves an accuracy rate of 91.6% in the field of e-government and also performs exceptionally well in scenarios like electronic medical question answering.

E. Related Works

This study [28] systematically evaluates and compares the performance of RAG and GraphRAG in general text tasks. The findings reveal the unique strengths of RAG and GraphRAG in question-answering and query-based text summarization tasks, while also highlighting the challenges in evaluating summarization tasks. These insights offer valuable guidance for future research directions. Additionally, this study proposes two strategies: Selection and Integration, to enhance the performance of question-answering tasks. It also suggests that future work could explore improving graph construction methods or developing new fusion strategies to combine the strengths of RAG and GraphRAG, thereby further enhancing the efficiency and effectiveness of the system.

HybridRAG [29] integrates the traditional vector RAG with the knowledge graph-based RAG to create a hybrid RAG architecture. By combining the advantages of KGs and LLMs, HybridRAG is able to efficiently extract and interpret complex information from unstructured financial text, achieving significant progress. In the financial domain, the HybridRAG system demonstrates outstanding performance in retrieval accuracy and answer generation, providing a more efficient and precise tool for financial analysis.

III. Design of Integration Strategies

In current digital age, enterprises have accumulated a vast amount of knowledge documents. Building intelligent QA systems based on these documents and LLMs has become an important trend for enterprises to embrace artificial intelligence. An efficient QA system can significantly enhance the efficiency of information retrieval within an enterprise, strengthen knowledge management capabilities, optimize customer service quality and experience, and provide strong support for enterprise decision-making and innovation. The application of enterprise-level QA systems has a profound impact on business operations. Therefore, providing users with accurate answers is the core requirement of enterprise QA systems.

Researches [22, 23, 28] have found that RAG performs exceptionally well in handling single-hop queries and queries that require detailed information, while KAG demonstrates strong capabilities in multi-hop queries that involve reasoning. As a result, RAG is more suitable for fact-based queries that rely on direct retrieval and the presentation of detailed information. In contrast, KAG is better suited for reasoning-type queries that involve the connection of multiple facts. It can effectively integrate information and conduct logical reasoning, thereby providing users with more precise and in-depth answers.

To better meet the needs of enterprises for QA systems, we propose five QA system construction strategies that integrates RAG and KAG. These strategies aim to provide diverse options for enterprises to build efficient and intelligent QA systems.

When integrating RAG and KAG to construct a QA system, two key issues need to be carefully considered. First, whether the enterprise documents should be stored in the RAG system or the KAG system, which will directly affect the system's retrieval efficiency and the way knowledge is utilized. Second, for the retrieval of user queries, whether to use the retrieval results from one of RAG or KAG, or to integrate the retrieval results from both systems to achieve more comprehensive and accurate information matching, is also a critical point that needs to be balanced.

Based on the above two core choices, we have designed multiple system construction strategies to meet the needs of enterprise QA systems. These strategies fully take into account the characteristics of enterprise data, application scenarios, and the requirements for the accuracy and efficiency of question answering. Through these strategies, enterprises can flexibly choose according to their own actual situations, thereby building an efficient QA system that best meets their business needs.

Strategy 1. In this strategy, enterprise documents are precisely stored in either the RAG or the KAG. Specifically, if the query requirement for a document is a direct retrieval of factual information, the document will be stored in the RAG to ensure quick and accurate acquisition of clear answers. Conversely, when the query involves complex multi-hop reasoning, the document will be stored in the KAG, leveraging its powerful associative reasoning capabilities to meet the needs of complex queries. The allocation of documents to either system can be achieved in two ways: through manual judgment and assignment by personnel with professional experience, or through intelligent classification using LLMs. When a user initiates a query, the QA system will intelligently select either the RAG or KAG for retrieval based on the classification of the query, with the process of constructing prompts drawing on relevant cutting-edge research [28].

The advantage of this strategy lies in its efficient use of storage resources. Since documents are stored in only one system, duplicate storage is avoided, thereby significantly saving storage space. In terms of retrieval efficiency, as each query requires only one search in either the RAG or KAG, the retrieval speed is greatly increased, effectively reducing user waiting time and enhancing user experience.

However, this strategy also presents some potential challenges. The selection of the document storage system requires either manual or automated judgment, and both manual allocation and model classification may introduce a certain error rate. On the other hand, accurate classification of query types is equally crucial during the query phase. Inaccurate classification may lead to unsatisfactory query results, thereby affecting the overall accuracy of the query.

Strategy 2. Same to Strategy 1, enterprise documents are classified and stored in either the RAG or KAG through manual means or LLMs. When a user initiates a query, both the RAG and KAG perform retrieval operations simultaneously. Subsequently, the system compares the two retrieval results and selects the one with a stronger relevance to the user's query to pass to the generator for generating the final response. Since the documents related to the query exist only in one of the RAG or KAG, there is no need to fuse the two retrieval results.

Like Strategy 1, this strategy stores documents in only one system, thereby saving some storage space. During retrieval, both the RAG and KAG participate, covering all documents in the QA system. This helps improve the accuracy of the responses to some extent. However, this strategy also has some drawbacks. First, storing documents requires manual or automatic judgment to determine whether they should be stored in RAG or KAG, which increases the complexity of management. Second, during querying, both RAG and KAG systems need to perform retrieval operations and select the query results, which inevitably increases the QA system's computational consumption and response time.

Strategy 3. In this strategy, enterprise documents are stored separately in RAG and KAG. When a user initiates a query, the LLM will precisely classify the query content based on specific prompt and then decide whether to allocate the query task to RAG or KAG for retrieval. After the retrieval is completed, the system will pass the results to the generator to produce the final response.

The advantage of this strategy lies in its comprehensive retrieval scope, which covers all enterprise documents and ensures that users can obtain as much information as possible. However, this approach also has some potential drawbacks. First, storing documents in both systems inevitably increases the demand for storage space, adding pressure to the management of the company's storage resources. Second, during the query process, the classification of the query content may introduce errors. If the classification is not accurate enough, it may lead to the query task being assigned to the wrong system, thereby reducing the accuracy of the final response.

Strategy 4. Enterprise documents are stored simultaneously in both RAG and KAG. During the query process, RAG and KAG independently execute retrieval operations. Subsequently, the system compares the two retrieval results and selects the more optimal result to pass to the generator, which then produces the final response. This selection process can be achieved by leveraging a LLM in conjunction with relevant prompt.

With this strategy, there is no need to pre-determine which system the enterprise documents should be stored in. The retrieval operation can cover all documents comprehensively, ensuring the integrity of the information. However, since both RAG and KAG need to perform searches, this inevitably increases system energy consumption. Moreover, to filter out the better search result, the system requires additional time for analysis and selection, thereby increasing the time it takes to generate the response.

Strategy 5. In this strategy, enterprise documents are stored in both RAG and KAG. When a user initiates a query request, both RAG and KAG will independently conduct retrieval. The retrieval results obtained from each system will then be deduplicated and merged. Subsequently, the merged results are passed to the generator to produce the final response.

This strategy has the advantage that enterprises do not need to choose which system to store documents. All documents can be fully covered during the retrieval, ensuring the completeness of information retrieval. However, there are also some drawbacks to this strategy. Since both RAG and KAG need to conduct retrieval independently, this inevitably increases system energy consumption. Moreover, the need to merge retrieval results from the two systems adds extra time, thereby extending the overall duration for the system to generate a response.

IV. Implementation Details and Experiments

A. Implementation Details

We developed a QA system that integrates RAG and KAG with above five strategies, providing users with a unified interaction interface to meet diverse QA needs.

The RAG system is implemented based on the open-source framework *Langchain* [30]. It utilizes *bge-large-zh-v1.5* [31] as the embedding model, *bge-rerank-large* [32] as the reranking model, and the efficient *FAISS* [33] as the vector database. When indexing enterprise documents, the chunk size is set to 1024 and the chunk overlap to 250 to optimize retrieval efficiency. During retrieval, the nearest neighbor search algorithm *similarity_search_by_vector* is employed, with the *top_k* parameter set to 10 to ensure the relevance and accuracy of the search results.

The KAG system is built on the open-source KAG framework from Ant Group. It also uses *bge-large-zh-v1.5* as the embedding model, with the maximum document segmentation length set to 1024. Other parameters are kept as the default settings of the KAG system.

In the generator component, we have used the powerful LLM *DeepSeek-R1-Distill-Qwen-32B* [8, 9], which provides efficient and accurate content generation capabilities for the system. This further enhances the overall performance and user experience of the QA system.

B. Experiments

To verify the effectiveness of the strategies, we constructed a dataset based on the enterprise documents of a power industry information and communication company, and assessed the accuracy of the answers through manual evaluation like these studies [34, 35].

We collected a total of 137 enterprise documents from the company to serve as the experimental datasets. In the experiment, we invited three domain experts familiar with the content of the enterprise documents to classify the documents. After classification, there were 78 fact-based documents and 59 inference-based documents. Subsequently, these three domain experts designed 50 query questions for the each type of documents, resulting in a total of 100 experimental queries.

After the QA system generated the query responses, the three domain experts evaluated the satisfaction of the answers. If two or more experts deemed the answer satisfactory, it was considered correct; otherwise, if fewer than two experts were satisfied, it was deemed incorrect.

C. Results

We first used the dataset to obtain answers from the RAG and KAG systems, and the results are shown in Table I. Then, we conducted QA tests on the five proposed strategies using the dataset, and the results are shown in Table II.

TABLE I. RESULTS OF RAG AND KAG IN Q&A

Methods	Accuracy
RAG	0.52
KAG	0.73

TABLE II. RESULTS OF INTEGRATION STRATEGIES IN Q&A

Methods	Accuracy
Strategy 1	0.57
Strategy 2	0.63
Strategy 3	0.68
Strategy 4	0.74
Strategy 5	0.78

From the all results, it can be seen that Strategy 5 has the highest accuracy rate, followed closely by Strategy 4. The accuracy rates of the other strategies fall within the range between the accuracy rates of RAG and KAG. Given that the current dataset contains a relatively limited number of questions, the performance of each strategy has certain reference value. However, further in-depth investigation is still needed through a broader range of well designed data.

V. CONCLUSION

In this paper, we explored the integration of RAG and KAG frameworks to enhance the accuracy and efficiency of enterprise QA systems. We proposed five strategies for combining RAG and KAG, each with its own advantages and trade-offs in terms of storage, retrieval efficiency, and response accuracy. Through the implementation and evaluation of these strategies using a dataset from a power industry information and communication company, we demonstrated that the integration of RAG and KAG can improve the performance of QA systems compared to using either framework alone. Future work may focus on developing more advanced techniques for document classification, optimizing retrieval algorithms, and exploring the use of hybrid models to achieve a better balance between accuracy and resource utilization.

ACKNOWLEDGMENT

This work was supported by State Grid Infommation & Telecommanication Branch, Science and Technology Project under grant (No.52993923000L).

REFERENCES

[1] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2025. A survey of large language models. arXiv preprint arXiv:2303.18223.

[2] Raza, M., Jahangir, Z., Riaz, M.B. et al. Industrial applications of large language models. Sci Rep 15, 13755 (2025). https://doi.org/10.1038/s41598-025-98483-1.

[3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. Advances in neural information processing systems 33 (2020), 1877 – 1901.

[5] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. arXiv preprint arXiv:2303.08774 (2023).

[6] feHugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. CoRR abs/2302.13971 (2023). https: //doi.org/10.48550/ARXIV.2302.13971 arXiv:2302.13971.

[7] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. ArXiv preprint abs/2307.09288 (2023). https://arxiv.org/abs/2307.09288.

[8] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. Preprint, arXiv:2309.16609.

[9] DeepSeek Team. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv. https://arxiv.org/pdf/2501.12948

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in Neural Information Processing Systems, 33:9459 – 9474, 2020.

[11] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms: Towards retrieval-augmented large language models. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pages 6491– 6501.

[12] Shailja Gupta, Rajesh Ranjan, and Surya Narayan Singh. "A Comprehensive Survey of Retrieval-Augmented Generation

(RAG): Evolution, Current Landscape and Future Directions", CoRR, abs/2410.12837, 2024.

[13] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, and Bin Cui. Retrieval-augmented generation for ai-generated content: A survey. arXiv preprint arXiv:2402.19473, 2024.

[14] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. Preprint, arXiv:2312.10997.

[15] Ran Xu, Wenqi Shi, Yue Yu, Yuchen Zhuang, Bowen Jin, May D Wang, Joyce C Ho, and Carl Yang. 2024. Ram-ehr: Retrieval augmentation meets clinical predictions on electronic health records. arXiv preprint arXiv:2403.00815.

[16] Nirmalie Wiratunga, Ramitha Abeyratne, Lasal Jayawardena, Kyle Martin, Stewart Massie, Ikechukwu NkisiOrji, Ruvan Weerasinghe, Anne Liret, and Bruno Fleisch. 2024. Cbr-rag: case-based reasoning for retrieval augmented generation in llms for legal question answering. In International Conference on CaseBased Reasoning, pages 445-460. Springer.

[17] Fatma Miladi, Valéry Psyché, and Daniel Lemire. 2024. Leveraging gpt-4 for accuracy in education: A comparative study on retrieval-augmented generation in moocs. In International Conference on Artificial Intelligence in Education, pages 427-434. Springer.

[18] Subash Neupane, Elias Hossain, Jason Keith, Himanshu Tripathi, Farbod Ghiasi, Noorbakhsh Amiri Golilarz, Amin Amirlatifi, Sudip Mittal, and Shahram Rahimi. "From Questions to Insightful Answers: Building an Informed Chatbot for University Resources", Computing Research Repository, abs/2405.08120, 2024.

[19] Boyu Zhang, Hongyang Yang, Tianyu Zhou, Muhammad Ali Babar, and Xiao-Yang Liu. 2023. Enhancing financial sentiment analysis via retrieval augmented large language models. In Proceedings of the fourth ACM international conference on AI in finance, pages 349-356.

[20] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2024. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv preprint arXiv:2311.05232.

[21] Matthew Dahl, Varun Magesh, Mirac Suzgun, and Daniel E Ho. 2024. Large legal fictions: Profiling legal hallucinations in large language models. arXiv preprint arXiv:2401.01301 (2024).

[22] H. Han, Y. Wang, H. Shomer, K. Guo, J. Ding, Y. Lei, M. Halappanavar, R. A. Rossi, S. Mukherjee, X. Tang et al., " Retrieval-augmented generation with graphs (graphrag), " arXiv preprint arXiv:2501.00309, 2024.

[23] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Junnan Dong, Hao Chen, Yi Chang, Xiao Huang, A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models. arXiv preprint arXiv:2501.13958, 2025.

[24] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. "GRAG: Graph Retrieval-Augmented Generation", North American Chapter of the Association for Computational Linguistics, pp. 4145-4157, 2025.

[25] Costas Mavromatis, and George Karypis. "GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning", Computing Research Repository, abs/2405.20139, 2024.

[26] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. arXiv preprint arXiv:2405.14831, 2024.

[27] Lei Liang, Mengshu Sun, Zhengke Gui, Zhongshu Zhu, Zhouyu Jiang, Ling Zhong, Yuan Qu, Peilong Zhao, Zhongpu Bo, Jin Yang, Huaidong Xiong, Lin Yuan, Jun Xu, Zaoyang Wang, Zhiqiang Zhang, Wen Zhang, Huajun Chen, Wenguang Chen, and Jun Zhou. "KAG: Boosting LLMs in Professional Domains Via Knowledge Augmented Generation", Computing Research Repository, abs/2409.13731, 2024.

[28] Haoyu Han, Harry Shomer, Yu Wang, Yongjia Lei, Kai Guo, Zhigang Hua, Bo Long, Hui Liu, and Jiliang Tang. "RAG Vs. GraphRAG: A Systematic Evaluation and Key Insights", CoRR abs/2502.11371, 2025.

[29] Bhaskarjit Sarmah, Benika Hall, Rohan Rao, Sunil Patel, Stefano Pasquali, and Dhagash Mehta. "HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction", Computing Research Repository: 608-616, 2024.

[30] LangChain. Applications that can reason. powered by langchain. [Online]. Available: https://www.langchain.com/.

[31] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-pack: Packaged resources to advance general chinese embedding. arXiv preprint arXiv:2309.07597.

[32] Bge-reranker-large, Bge-reranker-large: Model Card, Huggin g Face, 2025. [Online]. Available: https://huggingface.co/B AAI/bge-reranker-large.

[33] Facebook AI Research, "Faiss: Efficient similarity search," GitHub. 2024. [Online]. Available: https://github.com/faceb ookresearch/faiss.

[34] Xinyu Wang, Jijun Chi, Zhenghan Tai, Tung Sum Thomas Kwok, Muzhi Li, Zhuhong Li, Hailin He, Yuchen Hua, Peng Lu, Suyuchen Wang, Yihong Wu, Jerry Huang, Jingrui Tian, and Ling Zhou. "FinSage: A Multi-aspect RAG System for Financial Filings Question Answering", arXiv preprint arXiv 2504.14493, 2025.

[35] Tianyang Zhang, Zhuoxuan Jiang, Shengguang Bai, Tianrui Zhang, Lin, Yang Liu, and Jiawei Ren. "RAG4ITOps: A Supervised Fine-Tunable and Comprehensive RAG Framework for IT Operations and Maintenance", EMNLP 2024 Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, pp. 738-754, 2024.